Integración de Blockchain para la Trazabilidad y Auditoría en el Entrenamiento Distribuido de Modelos de Inteligencia Artificial

David Piotr Znojek
Francisco A. Pujol López
Departamento de Tecnología Informática y Computación
Escuela Politécnica Superior, Universidad de Alicante
Alicante, España
dawidznojek@gmail.com, fpujol@ua.es

***Resumen*—**El entrenamiento distribuido mediante Aprendizaje Federado (*Federated Learning*) permite preservar la privacidad de datos sensibles pero carece intrínsecamente de mecanismos para auditar la legitimidad de las contribuciones de los participantes, exponiendo al sistema a ataques de envenenamiento (*model poisoning*) con consecuencias críticas en sectores como el de los vehículos autónomos. En este artículo se presenta y evalúa una arquitectura híbrida que integra una red Blockchain permisionada basada en Hyperledger Besu y consenso IBFT 2.0 con el framework Flower y almacenamiento distribuido fuera de cadena mediante IPFS. Se diseña un contrato inteligente en Solidity que implementa un protocolo de doble verificación cruzada entre clientes y servidor agregador utilizando compromisos criptográficos SHA-256 de los pesos neuronales. La viabilidad del marco se valida empíricamente sobre un banco de pruebas distribuido entrenando un clasificador convolucional de señales y eventos viales bajo un riguroso protocolo experimental multiejecución (15 ejecuciones y 450 rondas federadas en total) bajo cinco configuraciones. Los resultados cuantitativos demuestran que, en el despliegue distribuido realista, la capa de trazabilidad añade una sobrecarga temporal reducida de tan solo el 9,5 % (6,34 s por ronda), manteniendo un consumo de gas estrictamente constante ($\mathcal{O}(1)$) y escalando con un impacto marginal (+0,39 s en mediana de ronda y +0,10 s en confirmación de bloque) al ampliar el quórum de 4 a 7 nodos validadores ($f = 1 \rightarrow f = 2$). Asimismo, la evaluación adversarial confirma una tasa de detección del 100 % frente a los vectores evaluados de adulteración e inyección no registrada en servidor y clientes con latencias de mitigación inferiores a 0,10 s (74 ms para aborto local y 44 ms para descarte en agregación). Esta investigación confirma que el coste de incorporar inmutabilidad y no repudio resulta plenamente asumible frente a las garantías de auditoría obtenidas.

***Palabras clave*—**Aprendizaje Federado, Blockchain, Hyperledger Besu, Trazabilidad, Consenso IBFT 2.0, IPFS, Auditoría Criptográfica.

I. Introducción

El auge de la Inteligencia Artificial (IA) y del Aprendizaje Automático (*Machine Learning*, ML) ha transformado sectores críticos como la salud, las finanzas y la conducción autónoma. No obstante, el entrenamiento de modelos de gran capacidad requiere volúmenes masivos de datos, frecuentemente sensibles o confidenciales (geolocalización, hábitos de consumo o registros biométricos). En los paradigmas tradicionales centralizados, la recopilación de información en un único repositorio introduce graves riesgos de privacidad, problemas de cumplimiento normativo (como el RGPD) y cuellos de botella computacionales y energéticos.

El Aprendizaje Federado (*Federated Learning*, FL) [1] surgió como una alternativa descentralizada en la que múltiples nodos o clientes entrenan copias locales del modelo con sus propios datos privados, transmitiendo a un servidor central únicamente los parámetros o gradientes actualizados. Aunque FL solventa en gran medida la fuga de datos crudos, introduce una vulnerabilidad estructural: la falta de confianza intrínseca entre los participantes. En un entorno distribuido no supervisado, actores maliciosos o nodos comprometidos pueden ejecutar ataques de envenenamiento (*data poisoning* o *model poisoning*) inyectando pesos manipulados. Por ejemplo, en el contexto de los Vehículos Conectados y Autónomos (CAV), una pequeña alteración coordinada en los pesos podría inducir al modelo global a no clasificar correctamente una señal de tráfico crítica (como un STOP), con consecuencias potencialmente fatales.

Para mitigar este riesgo, la tecnología *Blockchain* ofrece una solución idónea como registro contable inmutable, descentralizado y verificable [2, 3]. Al integrar Blockchain en un flujo de entrenamiento distribuido, cada actualización de parámetros, ronda de agregación e identidad de los clientes queda registrada criptográficamente mediante contratos inteligentes (*smart contracts*), habilitando la trazabilidad absoluta y la no repudiación de las contribuciones.

Sin embargo, esta convergencia plantea un compromiso técnico fundamental: el dilema entre **seguridad y ren-**

dimiento. Mientras que la Blockchain garantiza inmutabilidad y consenso bizantino, introduce una sobrecarga inevitable de latencia de red, tiempo de confirmación de transacciones y consumo de recursos. En aplicaciones de misión crítica o en tiempo cuasi-real, resulta imperativo responder a la siguiente pregunta: *¿cuánta latencia y coste computacional estamos dispuestos a asumir a cambio de un entrenamiento auditable y más resistente a manipulaciones?*

A. Motivación y Trabajos Previos

Este trabajo se fundamenta y amplía dos líneas de investigación desarrolladas en la Universidad de Alicante:

1. **Verificación de eventos vehiculares en tiempo real** [4]: demostró la viabilidad de una red privada Hyperledger Besu con consenso IBFT 2.0 para registrar alertas de tráfico con latencias de 0,32 a 0,82 segundos, mejorando el tiempo de respuesta humano.
2. **Framework FL-Blockchain** [5]: planteó conceptualmente el uso de contratos inteligentes para registrar los *hashes* criptográficos de los pesos de entrenamiento en cada ronda federada.

La presente investigación valida, formaliza e implementa experimentalmente la convergencia de ambas propuestas en un banco de pruebas real, evaluando tanto la integridad como el impacto en el rendimiento temporal.

B. Contribuciones Principales

Las contribuciones de este artículo se resumen en los siguientes puntos:

- **Diseño de una arquitectura híbrida**: Se formula un marco integral que acopla una red permisionada basada en Hyperledger Besu con el framework de aprendizaje federado Flower [6] y un sistema de almacenamiento distribuido direccionable por contenido (IPFS).
- **Trazabilidad e integridad mediante Smart Contracts**: Se diseña un contrato inteligente en Solidity que orquesta la verificación cruzada de modelos mediante resúmenes SHA-256, registrando de forma inalterable las contribuciones de cada cliente y el modelo global consolidado por ronda.
- **Evaluación empírica y mitigación de ataques adversariales**: Se somete la infraestructura a vectores de ataque deliberados de inyección de pesos corruptos (tanto en servidor comprometido como en clientes envenenadores), validando la capacidad del protocolo de doble verificación para detectar y neutralizar de forma determinista cualquier discrepancia de integridad o modelo no registrado on-chain (alcanzando una tasa de éxito del 100 % en los vectores de adulteración evaluados) con tiempos de reacción inferiores a 0,1 segundos.
- **Evaluación cuantitativa de la infraestructura**: Se despliega un clasificador convolucional de tráfico con el propósito de emplear la estructura de aprendizaje federado como banco de pruebas experimental para extraer métricas cuantitativas reproducibles (sobrecoste temporal, consumo de gas y escalabilidad de validadores), delimitando la optimización fina del modelo como trabajo futuro.
- **Análisis de viabilidad operativa**: Se responde analíticamente al dilema latencia-seguridad, delimitando los escenarios en los que la sobrecarga del registro en cadena resulta plenamente asumible frente a esquemas no auditables.

Con el fin de garantizar la reproducibilidad y la transferencia científica, el código fuente completo del framework, los contratos inteligentes en Solidity, las plantillas de despliegue de la red Hyperledger Besu con Docker Compose y los scripts de experimentación se encuentran publicados de forma abierta en el repositorio de GitHub: `https://github.com/dpz6-ua/Blockchain_ML`.

C. Estructura del Artículo

El resto del manuscrito se estructura de la siguiente manera: la Sección II expone el estado del arte y fundamentos teóricos de Blockchain y Aprendizaje Federado. La Sección III detalla la arquitectura propuesta y el diseño del contrato inteligente. La Sección IV describe el banco de pruebas experimental y el dataset empleado. La Sección V presenta y discute los resultados de rendimiento y seguridad. Finalmente, la Sección VI resume las conclusiones y líneas futuras.

II. Fundamentos Teóricos y Trabajos Relacionados

La arquitectura propuesta converge tres pilares tecnológicos: registros distribuidos permisionados con consenso BFT, almacenamiento direccionable por contenido y aprendizaje federado. A continuación se sintetizan sus fundamentos teóricos y las limitaciones detectadas en la literatura.

A. Blockchain Permisionada y Consenso IBFT 2.0

Una cadena de bloques es una estructura de datos inmutable y distribuida en la que las transacciones se agrupan en bloques encadenados mediante funciones hash criptográficas resistentes a colisiones (como SHA-256) [3, 7].

Frente a las redes públicas (*permissionless*) como Bitcoin o Ethereum Mainnet, penalizadas por altas latencias y costes económicos de transacción, los consorcios industriales adoptan redes permisionadas basadas en la Máquina Virtual de Ethereum (EVM) [8], tales como Hyperledger Besu [9].

En estos entornos, los mecanismos basados en consumo de recursos (PoW) o capital (PoS) se sustituyen por algoritmos de tolerancia a fallos bizantinos. El protocolo **IBFT 2.0** (*Istanbul Byzantine Fault Tolerant*) [10] estructura el consenso en torno a un conjunto predefinido de validadores que alternan el rol de proponente mediante un esquema rotatorio (*round-robin*). Un bloque solo se anexa a la cadena tras una votación en tres fases (pre-prepare, prepare, commit) firmada por una supermayoría de al menos dos tercios:

$$n \geq 3f + 1 \qquad ((1))$$

donde n denota el número total de nodos validadores y f el número máximo de participantes defectuosos o bizantinos tolerables.

IBFT 2.0 ofrece una propiedad determinante frente a otros consensos: la **finalidad determinista inmediata**. Al carecer de bifurcaciones probabilísticas (*forks*), una vez que una ronda de entrenamiento o contribución se sella en un bloque, es irrefutable y permanente. Asimismo, aunque el precio del gas (*gas price*) se configure en cero para anular tarifas monetarias, el cómputo de *gas used* [11] se mantiene como métrica objetiva del coste de cómputo y almacenamiento impuesto a la red.

B. Almacenamiento Distribuido Off-Chain (IPFS)

El almacenamiento directo de redes neuronales profundas en una Blockchain es inviable debido al tamaño de los pesos (decenas o cientos de megabytes), lo cual provocaría una saturación crítica del estado de la EVM (*state bloat*) y un consumo prohibitivo de gas.

Para superar este cuello de botella, se implementa el paradigma de **almacenamiento fuera de cadena con compromiso dentro de cadena** (*off-chain storage with on-chain commitment*) utilizando el Sistema de Archivos Interplanetario (IPFS) [12]. IPFS opera sobre una topología P2P basada en direccionamiento por contenido (*content addressing*): cada archivo se identifica de forma unívoca mediante un *Content Identifier* (CID), obtenido a partir del hash criptográfico de su contenido. Al registrar únicamente el CID y el hash SHA-256 en el contrato inteligente, se asegura que cualquier descarga posterior sea verificada bit a bit contra el registro inmutable, garantizando deduplicación, disponibilidad y verificación formal de integridad con coste computacional on-chain mínimo.

C. Aprendizaje Federado y Algoritmo FedAvg

El Aprendizaje Federado (FL) [1] resuelve la preservación de la privacidad de datos crudos al desacoplar el entrenamiento local de la agregación global. Entre los algoritmos de consenso federado, **FedAvg** (*Federated Averaging*) representa el estándar fundacional. En cada ronda de comunicación t, el servidor distribuye el vector de pesos global actual $w^{(t)}$ a un subconjunto de K clientes seleccionados. Cada cliente entrena el modelo sobre su partición local $\mathcal{D}_k$ minimizando una función de pérdida, obteniendo los parámetros locales $w_k^{(t)}$. El servidor calcula el nuevo modelo global consolidado como la media ponderada del volumen de muestras locales n_k:

$$w^{(t+1)} = \sum_{k=1}^{K} \frac{n_k}{n} w_k^{(t)}, \quad \text{donde } n = \sum_{k=1}^{K} n_k \qquad ((2))$$

El marco de referencia **Flower** [6] proporciona una infraestructura desacoplada y eficiente para orquestar este flujo mediante llamadas a procedimientos remotos bidireccionales sobre gRPC.

D. Vulnerabilidades en FL y Trabajos Relacionados

A pesar de sus ventajas de privacidad, los sistemas FL tradicionales sufren de severas vulnerabilidades de seguridad:

1. **Servidor central como punto único de fallo y confianza**: No existe salvaguarda para verificar si el servidor manipuló la ponderación de FedAvg o si inyectó sesgos maliciosos en $w^{(t+1)}$.
2. **Ataques de envenenamiento y falta de trazabilidad**: Nodos malintencionados pueden realizar *model poisoning* o inyectar puertas traseras (*backdoors*). Sin un historial no repudiable de identidades y hashes, resulta imposible deslindar qué participante contaminó el modelo global.

Diversas aproximaciones en la literatura han explorado la hibridación de Blockchain y FL. No obstante, gran parte de los trabajos teóricos sufren de problemas de escalabilidad al pretender ejecutar la agregación matemática completa dentro de la EVM o depender de redes públicas costosas. En el dominio de los sistemas de transporte inteligentes (ITS), Pujol et al. [4] demostraron que Hyperledger Besu con IBFT 2.0 ofrece latencias subsegundo viables para la cooperación vehicular, y CloudLab-AIA [5] conceptualizó el registro de hashes de modelos.

Para contextualizar el aporte de esta investigación frente a las principales propuestas de la literatura especializada, la Tabla I sintetiza una comparativa cualitativa y arquitectural. A diferencia de esquemas basados en consenso PoW o de almacenamiento on-chain (como BlockFL), o de

Cuadro I: Comparativa cualitativa y arquitectural frente a los principales marcos de Blockchain-FL de la literatura.

Propuesta / Framework	Consenso Blockchain	Almacenamiento Modelo	Protocolo Verificación	Coste Transaccional	Tolerancia a Fallos
BlockFL (Kim et al., 2019)	PoW (Lento / Ineficiente)	On-Chain (Cuello de botella)	Unidireccional	Criptomoneda volátil	Probabilística ($< 50\,\%$)
FedChain (Zhai et al., 2021)	Raft (No bizantino)	Off-Chain centralizado	Sin verificación previa	Red privada	Solo caídas (*crash*, $f < N/2$)
BFL Vehicular (Pujol et al., 2024) [4]	IBFT 2.0 (Permisionado)	Registro de alertas IoT	Monitoreo de tráfico	Libre de gas comercial	Bizantina formal ($N \geq 3f + 1$)
Nuestra Arquitectura	**QBFT / IBFT 2.0 (Besu)**	**Híbrido Off-Chain (IPFS DHT)**	**Doble Cruzada (Asc./Desc.)**	**Gas determinista** $\mathcal{O}(1)$	**Bizantina formal** ($f = 1 \rightarrow 2$)

aquellos que delegan el consenso en tolerancias a fallos no bizantinas (como FedChain con Raft), nuestra arquitectura articula una doble verificación cruzada bidireccional sobre una red permisionada de consorcio con finalidad inmediata y coste computacional predecible $\mathcal{O}(1)$.

III. Arquitectura del Sistema Propuesto

Para dotar al aprendizaje federado de trazabilidad y resistencia frente a comportamientos bizantinos, se propone una arquitectura híbrida desacoplada en tres planos operacionales: el plano de computación federada (Flower), el plano de almacenamiento direccionable fuera de cadena (IPFS) y el plano de consenso e inmutabilidad dentro de cadena (Hyperledger Besu). La Figura 1 ilustra la interacción integral entre estos componentes.

A. Diseño del Smart Contract: FLRegistry

El componente de gobernanza y auditoría lógica se implementa mediante el contrato inteligente `FLRegistry`, codificado en Solidity (versión `0.8.10`). Su misión es registrar un historial criptográfico determinista de cada ronda sin albergar matrices numéricas masivas on-chain.

1) Estructuras de datos y persistencia: La entidad nuclear del contrato es la estructura `ModelRound`, definida como:

```
struct ModelRound {
    uint256 roundNum;
    string modelHash;
    address author;
    uint256 timestamp;
}
```

donde `roundNum` indexa la iteración federada, `modelHash` almacena el CID o digest SHA-256 del artefacto del modelo, `author` guarda la dirección pública Ethereum del emisor (`msg.sender`) y `timestamp` captura la marca temporal del bloque validado.

El estado del contrato se organiza mediante dos tablas hash (*mappings*) complementarias:

- `mapping(uint256 =>ModelRound) public history`: almacena de forma secuencial los modelos globales agregados por el servidor al término de cada ronda t.
- `mapping(uint256 =>mapping(address => ModelRound)) public clientHistory`: indexa las contribuciones locales individuales de cada cliente en cada ronda, posibilitando la auditoría forense granular de las participaciones.

Adicionalmente, se mantienen los punteros de estado `latestRound` y el indicador booleano `hasModel` para optimizar lecturas directas $\mathcal{O}(1)$ sin requerir iteraciones sobre el almacenamiento persistente.

2) Primitivas de escritura y autenticación: El contrato expone dos métodos transaccionales que alteran el estado y emiten el evento desacoplado `ModelUpdated`:

- `updateModel(uint256 round, string memory hash)`: Invocado operacionalmente por el servidor tras converger el algoritmo de agregación. A nivel de diseño del banco de pruebas, la función se implementa con visibilidad pública para desacoplar el contrato de dependencias estáticas de despliegue; no obstante, la EVM registra de forma inmutable la dirección emisora en el campo `author` (`msg.sender`). Esto permite a los clientes verificar criptográficamente frente a la clave pública autorizada del servidor que el modelo global sellado proviene legítimamente del agregador del consorcio.

3) Primitivas de lectura y verificación: Para no sobrecargar la red con costes de ejecución, las consultas de verificación se implementan como funciones `view`, las cuales se resuelven localmente en el nodo RPC sin consumir gas:

- `getLastModel()`: Permite a cualquier cliente recuperar el CID del último modelo global consolidado y su marca de tiempo.
- `getClientModel(uint256 round, address client)`: Permite al servidor y a auditores externos cotejar el CID reportado por un cliente frente a su registro on-chain firmado.

B. Flujo Operacional y Doble Verificación Cruzada

El ciclo de vida del entrenamiento federado se estructura en una fase de inicialización única seguida de un protocolo iterativo con verificación bidireccional estricta, como se resume a continuación:

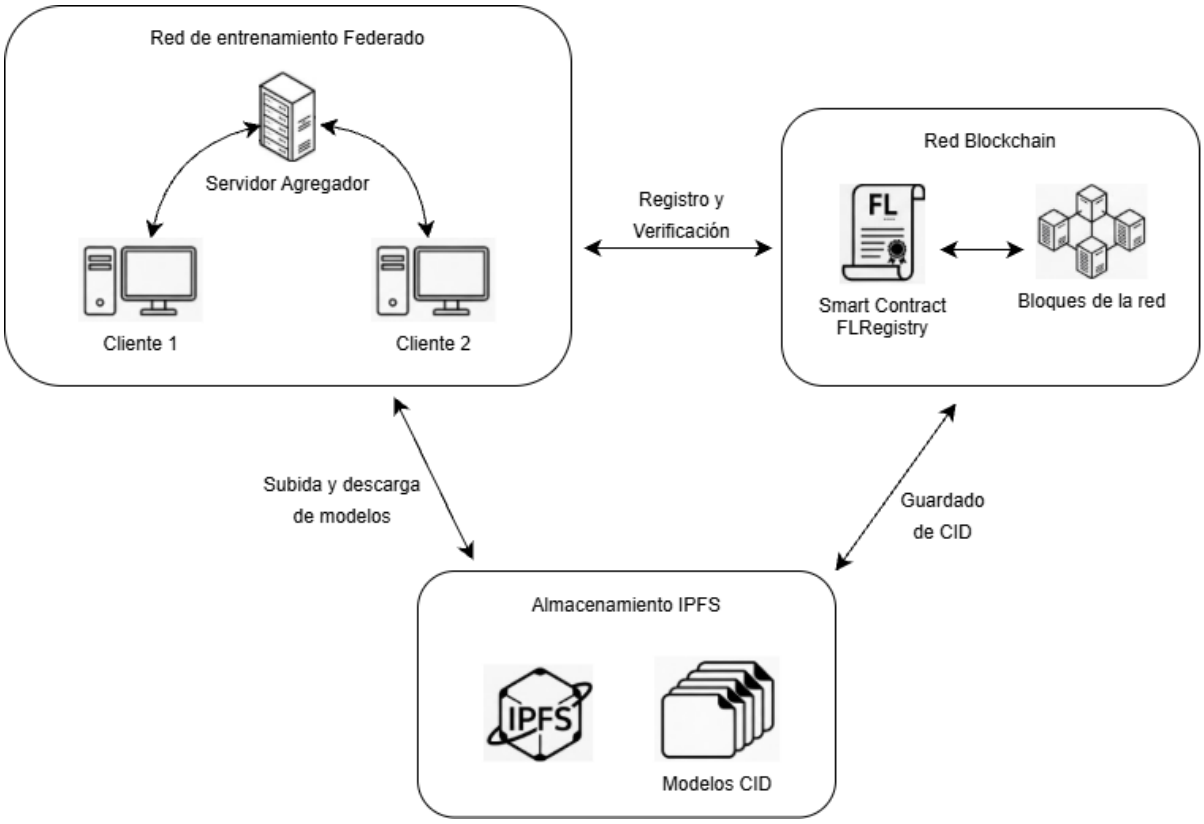


Figura 1: Arquitectura general del sistema híbrido: desacoplamiento entre el cómputo federado (Flower), el almacenamiento de pesos fuera de cadena (IPFS) y el registro inmutable de trazabilidad (Hyperledger Besu con IBFT 2.0).

1) Fase 0: Arranque y Modelo Base (Ronda 0): Antes de iniciar la concurrencia:

1. Se inicializa la red Besu con consenso IBFT 2.0 y el daemon IPFS.
2. Se despliega `FLRegistry` registrando su dirección y ABI en un descriptor JSON distribuido.
3. El servidor instancia los pesos iniciales $w^{(0)}$, los serializa y los almacena en IPFS, recibiendo un CID inicial CID_0.
4. El servidor ejecuta `updateModel(0,` CID_0`)`, fijando el punto de partida auditable para todos los participantes.

2) Fase 1: Verificación Descendente (Cliente protege frente a Servidor): Al recibir la señal de inicio de ronda junto con los pesos candidatos $w_{cand}^{(t)}$ vía gRPC (Figura 2):

1. El cliente consulta la Blockchain invocando `getLastModel()` y obtiene el $\text{CID}_{reg}^{(t-1)}$ oficial.
2. El cliente descarga el modelo referenciado desde IPFS y realiza una verificación bit a bit tensor por tensor frente a $w_{cand}^{(t)}$.
3. Si se detecta cualquier discrepancia entre los pesos entregados y el compromiso sellado en la cadena, el cliente **aborta el entrenamiento inmediatamente**, neutralizando ataques donde un servidor comprometido intente inyectar modelos sesgados selectivamente a clientes concretos.

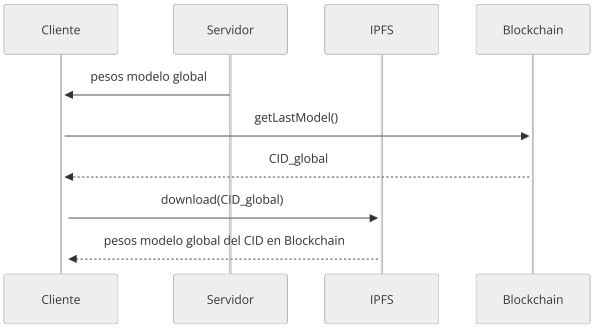


Figura 2. Fase 1: Verificación descendente en el cliente: cotejo del modelo recibido frente al CID registrado on-chain y descargado de IPFS.

3) Fase 2: Entrenamiento Local y Registro de Contribución: Si la comprobación previa es satisfactoria (Figura 3):

1. El cliente ejecuta el entrenamiento local con su partición privada de datos.
2. Serializa los nuevos pesos locales $w_k^{(t)}$ y los almacena en IPFS, obteniendo $\text{CID}_k^{(t)}$.
3. Emite una transacción firmada con su clave privada invocando `registerClientModel(`t, $\text{CID}_k^{(t)}$`)` y aguarda el recibo (*receipt*) de inclusión en el bloque IBFT 2.0.
4. Retorna a Flower los parámetros junto con el identificador $\text{CID}_k^{(t)}$ y su clave pública.

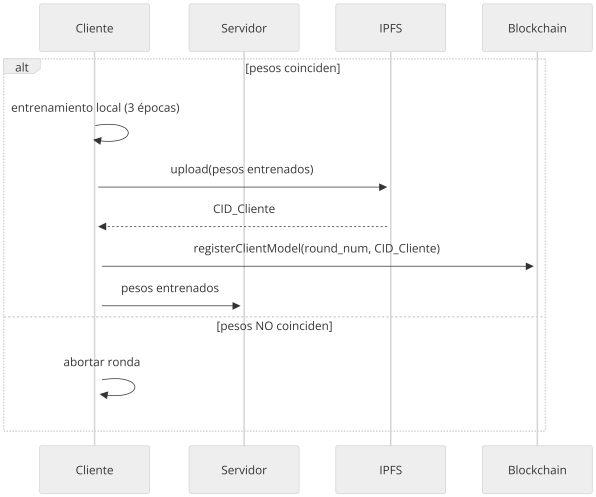


Figura 3. Fase 2: Entrenamiento local en el cliente, subida de pesos a IPFS y transacción firmada a Hyperledger Besu.

4) Fase 3: Verificación Ascendente y Agregación Global: Al recibir los pesos de los clientes en el servidor agregador (Figura 4):

1. Para cada cliente k, el servidor consulta `getClientModel(`t, address_k`)` en la Blockchain.
2. Comprueba que el CID reportado por gRPC coincida con el hash inmutable registrado y firmado por el cliente. Si la comprobación falla, la contribución es descartada de forma preventiva.
3. Aplica la agregación ponderada FedAvg únicamente sobre las contribuciones debidamente auditadas:

$$w^{(t+1)} = \sum_{k \in \mathcal{K}_{valid}} \frac{n_k}{n_{valid}} w_k^{(t)} \qquad ((3))$$

4. Sube el nuevo modelo consolidado a IPFS, obteniendo $\text{CID}_{global}^{(t+1)}$, y lo sella en Besu invocando `updateModel(`$t+1$, $\text{CID}_{global}^{(t+1)}$`)`.

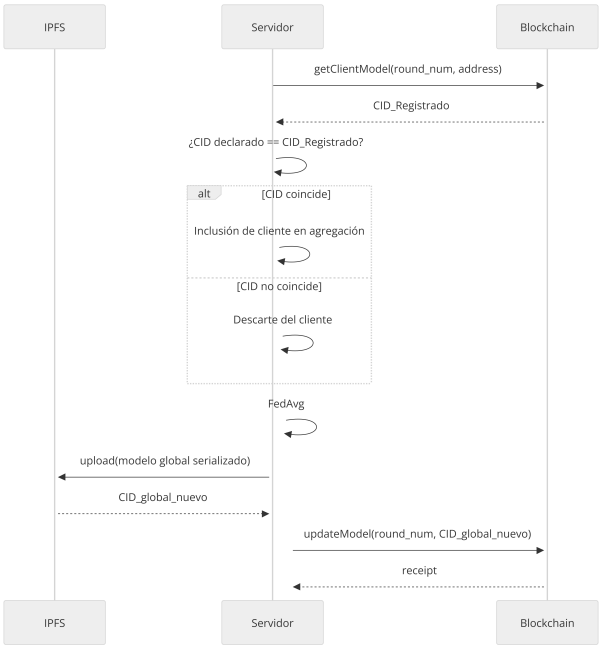


Figura 4. Fase 3: Verificación ascendente en el servidor: validación de CIDs on-chain, agregación FedAvg y sellado global de ronda.

Este mecanismo garantiza trazabilidad completa de extremo a extremo, no repudio y aislamiento de participantes disidentes.

IV. Implementación y Entorno Experimental

Con el fin de aislar y evaluar cuantitativamente el sobrecoste introducido por la capa de trazabilidad y auditoría, se desplegó un banco de pruebas experimental representativo de un entorno de computación distribuida vehicular.

A. Infraestructura de Hardware y Software

La evaluación empírica se llevó a cabo utilizando dos estaciones físicas interconectadas en la misma red de área local (LAN), cuyas especificaciones técnicas se detallan en la Tabla II:

- **Estación 1 (Nodo Central)**: Aloja el servidor agregador Flower, el nodo RPC principal de la red Besu y el nodo de almacenamiento IPFS.
- **Estación 2 (Nodos de Borde / Clientes)**: Aloja y ejecuta los clientes de entrenamiento federado (*members* 2 y 3).

Cuadro II
Especificaciones del banco de pruebas experimental.

Parámetro	Estación 1 (Servidor/RPC)	Estación 2 (Clientes)
Procesador	Intel Core i5-1135G7	Intel Core i7-9700
Frecuencia base	2,40 GHz	3,00 GHz
Núcleos / Hilos	4 núcleos / 8 hilos	8 núcleos / 8 hilos
Memoria RAM	7,6 GB (asignada en WSL2)	8,0 GB
Sistema Operativo	Ubuntu 22.04 LTS (WSL2)	Windows 11

El stack de software está compuesto por Hyperledger Besu con soporte de transacciones privadas mediante Tessera, orquestado mediante Docker Compose; el daemon IPFS Kubo (`v0.32.1`); Python `3.12` con PyTorch para la inferencia y optimización neuronal; Flower (`flwr`) para la coordinación gRPC de la federación; y la biblioteca `web3.py` para la conectividad JSON-RPC con la EVM. Todos los contratos inteligentes, configuraciones de red y scripts de orquestación se encuentran disponibles en el repositorio público `https://github.com/dpz6-ua/Blockchain_ML`.

Las dos estaciones se encuentran comunicadas a través de un enlace de red de área local inalámbrico (Wi-Fi 802.11ac a 5 GHz), registrando una latencia media de ida y vuelta (*RTT ping*) de $3{,}8 \pm 1{,}1$ ms y un ancho de banda sostenido de 100 Mbps. El tamaño del modelo neuronal serializado (`NetCliente`) es de exactamente 1,95 MB en formato binario. Para garantizar un rendimiento óptimo en la capa de almacenamiento distribuido, la subida a IPFS se efectúa mediante la API HTTP en el endpoint `/api/v0/add?pin=false`, lo que permite registrar y generar el identificador CID en memoria local en milisegundos.

B. Modelo Neuronal y Conjunto de Datos Vehicular

Como caso de uso para simular un sistema de asistencia a la conducción en vehículos conectados y autónomos (CAV), se adoptó el dataset de clasificación de eventos de tráfico del grupo CloudLab-AIA [5].

El problema consiste en una tarea de clasificación multiclase orientada a la detección en carretera con cuatro categorías:

1. Accidente vial (`accident`).
2. Congestión o tráfico denso (`dense_traffic`).
3. Condiciones meteorológicas adversas (`meteorology`).
4. Flujo de tráfico fluido (`sparse_traffic`).

El conjunto de datos comprende un total de 3.415 imágenes distribuidas en: meteorología adversa (1.027), tráfico fluido (947), tráfico denso (778) y accidentes (663). Las imágenes se preprocesan mediante reescalado a una resolución de 32×32 píxeles en tres canales cromáticos (RGB) y normalización estándar. Para preservar la comparabilidad estadística y eliminar sesgos de distribución, el conjunto de datos se reparte entre los $K = 2$ clientes según un esquema independiente e idénticamente distribuido (IID).

El modelo implementado (`NetCliente`) es una red neuronal convolucional (CNN) estructurada en dos bloques convolucionales consecutivos (con kernels de 3×3, función de activación ReLU y submuestreo por *Max Pooling*), seguidos de dos capas densas completamente conectadas (*fully connected*) que proyectan el vector de características hacia las 4 neuronas de salida. El entrenamiento se realiza mediante la función de pérdida de entropía cruzada (*Cross-Entropy Loss*) y el algoritmo de optimización Adam con una tasa de aprendizaje $\eta = 10^{-3}$, configurando un tamaño de lote (*batch size*) de 16 muestras en entrenamiento y 32 en evaluación, 3 épocas locales de entrenamiento por ronda y un horizonte total de 30 rondas federadas.

Es importante remarcar que el propósito primordial de este estudio no radica en diseñar el modelo más óptimo ni en maximizar su precisión de clasificación, sino en utilizar la arquitectura de aprendizaje federado como un banco de pruebas realista para generar y analizar métricas cuantitativas del sistema (sobrecoste temporal, escalabilidad y consumo de gas). La optimización del modelo (ajuste de hiperparámetros, mitigación del sobreajuste o empleo de arquitecturas más profundas) se plantea deliberadamente como trabajo futuro, actuando aquí la red neuronal como una carga de trabajo experimental de referencia.

C. Matriz de Configuraciones Experimentales

Para desglosar y desacoplar de forma analítica el impacto individual de cada componente, se diseñaron cinco configuraciones experimentales (E1 a E5), resumidas en la Tabla III.

Cuadro III
Matriz de configuraciones experimentales evaluadas.

ID Exp.	Estaciones	Capa Blockchain	Validadores	Objetivo
E1	1	Sí	4	Sobrecarga BC local
E2	1	No	—	Baseline local
E3	2	No	—	Baseline distribuido
E4	2	Sí	4	Sobrecarga BC distribuida
E5	2	Sí	7	Escalabilidad IBFT 2.0

Este diseño metodológico permite efectuar comparaciones controladas de una sola variable:

- **Coste directo de Blockchain**: Comparación de E1 vs. E2 en entorno monomáquina, y de E4 vs. E3 en entorno distribuido en red.

- **Efecto de la distribución física**: Comparación de E2 vs. E3 (sin Blockchain) y de E1 vs. E4 (con Blockchain de 4 validadores).
- **Escalado del consenso BFT**: Comparación de E4 (4 validadores) vs. E5 (7 validadores), evaluando la penalización de latencia al elevar la tolerancia a fallos de $f = 1$ a $f = 2$.

V. Evaluación Experimental y Resultados

En esta sección se analiza cuantitativamente el comportamiento del sistema a lo largo del banco de pruebas experimental, respondiendo a las preguntas de investigación formuladas: el sobrecoste temporal de la capa de auditoría, el desacoplamiento de recursos físicos, la escalabilidad del consenso bizantino IBFT 2.0, la resiliencia empírica frente a ataques de adulteración y la previsibilidad del consumo de gas en la EVM. Con el objetivo de garantizar solidez y significancia estadística, cada una de las cinco configuraciones experimentales (E_1 a E_5) fue evaluada a lo largo de 3 ejecuciones independientes completas de 30 rondas cada una, totalizando un corpus de 15 ejecuciones y 450 rondas federadas ($N_{\text{total}} = 450$).

A. Convergencia del Modelo y Análisis de Pérdida

La Figura 5 ilustra la evolución de la función de pérdida de entropía cruzada en el conjunto de validación durante las 30 iteraciones federadas.

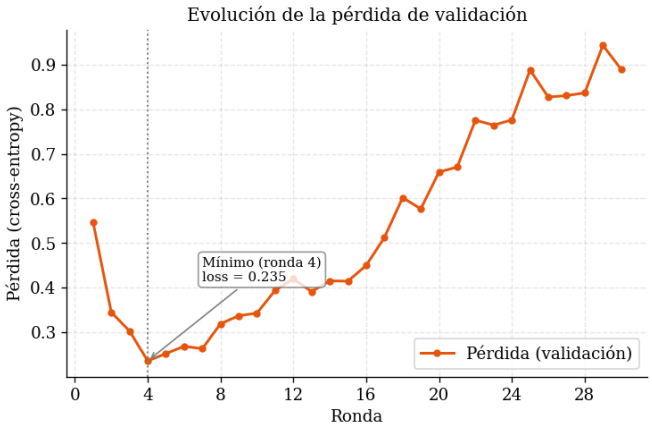


Figura 5. Evolución de la pérdida de validación (*cross-entropy*) a lo largo de las 30 rondas federadas.

El modelo muestra un rápido aprendizaje inicial, alcanzando un mínimo de pérdida óptimo en la ronda 4 ($\mathcal{L} \approx 0{,}24$) con una precisión de clasificación que se estabiliza en el 97 %. A partir de dicho punto, la pérdida experimenta un incremento gradual y sostenido hasta situarse en torno a 0,90 hacia el final del ciclo. Este comportamiento representa un caso paradigmático de sobreajuste (*overfitting*) provocado por el régimen de 90 épocas acumuladas (3 épocas locales × 30 rondas federadas) sobre un conjunto de entrenamiento acotado, donde la red incrementa excesivamente su confianza en predicciones.

El mínimo de pérdida en la ronda 4 sugiere que la incorporación de técnicas como la parada temprana (*early stopping*) habría detenido el entrenamiento en ese instante, logrando una versión más ajustada del clasificador. Sin embargo, se mantuvieron de forma deliberada las 30 rondas predefinidas porque el objetivo central de esta investigación no es optimizar el modelo de IA ni perseguir su punto de máxima eficiencia, sino someter la infraestructura de aprendizaje federado a una carga constante para extraer métricas cuantitativas reproducibles (latencia por ronda, tiempo de confirmación y coste de gas). De este modo, la optimización fina del modelo se pospone como trabajo futuro.

Es fundamental enfatizar que **la curva de convergencia es estrictamente idéntica con o sin la presencia de Blockchain**. Dado que la capa de trazabilidad opera exclusivamente en la verificación e indexación criptográfica fuera de cadena y no interfiere en los tensores ni en los gradientes de FedAvg, la dinámica de entrenamiento se mantiene inalterada.

B. Formalización Matemática de Métricas de Rendimiento

Con el fin de aislar de forma unívoca la latencia impuesta por cada componente del sistema, el tiempo total por ronda en los nodos de borde (T_{client}) y en el servidor agregador (T_{server}) en la iteración t se define analíticamente como:

$$T_{\text{client}}^{(t)} = t_{\text{verif_desc}}^{(t)} + t_{\text{train}}^{(t)} + t_{\text{ipfs_up}}^{(t)} + t_{\text{tx_besu}}^{(t)} \quad ((4))$$

$$T_{\text{server}}^{(t)} = t_{\text{wait}}^{(t)} + t_{\text{verif_asc}}^{(t)} + t_{\text{fedavg}}^{(t)} + t_{\text{ipfs_global}}^{(t)} + t_{\text{block_confirm}}^{(t)} \quad ((5))$$

donde $t_{\text{verif_desc}}$ representa la comprobación local contra el CID de Besu; t_{train}, el tiempo de cálculo de gradientes con PyTorch; $t_{\text{ipfs_up}}$, la publicación binaria de pesos; $t_{\text{tx_besu}}$, la emisión y confirmación de `registerClientModel`; t_{wait}, la ventana de recepción gRPC; $t_{\text{verif_asc}}$, la validación en `getClientModel`; t_{fedavg}, la agregación matricial; y $t_{\text{block_confirm}}$, la confirmación del bloque IBFT 2.0 tras invocar `updateModel`.

Para medir el sobrecoste relativo (Δ_{overhead}) entre una configuración con seguridad E_{BC} y su respectiva línea base E_{base}, se emplea la mediana de las distribuciones:

$$\Delta_{\text{overhead}} = \frac{\text{Med}(T_{E_{\text{BC}}}) - \text{Med}(T_{E_{\text{base}}})}{\text{Med}(T_{E_{\text{base}}})} \times 100 \quad [\,\%] \quad ((6))$$

Asimismo, la reproducibilidad y dispersión estadística se caracterizan mediante el Error Estándar de la Media (SE):

$$SE = \frac{\sigma}{\sqrt{N_{\text{rounds}}}} \qquad ((7))$$

donde σ denota la desviación típica muestral y $N_{\text{rounds}} = 90$ el total de observaciones agregadas por configuración (30 rondas $\times 3$ ejecuciones independientes).

C. Visión General del Rendimiento Temporal Consolidado

La Tabla IV sintetiza el tiempo total requerido por ronda federada en las cinco configuraciones experimentales, consolidando las tres ejecuciones independientes (90 rondas por configuración) y reportando la mediana, media, desviación típica (σ) y error estándar (SE). Asimismo, la Figura 6 refleja la serie temporal a lo largo de las rondas.

CUADRO IV
TIEMPO TOTAL DE EJECUCIÓN POR RONDA FEDERADA CONSOLIDADO (3 EJECUCIONES, 90 RONDAS POR CONFIGURACIÓN, $N_{\text{TOTAL}} = 450$).

Exp.	Máq.	Configuración	Runs	Mediana (s)	Media (s)	σ (s)	SE (s)
E_1	1	4 val · con BC	3	**85,42**	97,09	27,64	2,91
E_2	1	Baseline sin BC	3	**75,70**	76,45	2,83	0,30
E_3	2	Baseline sin BC	3	**66,62**	67,62	3,76	0,40
E_4	2	4 val · con BC	3	**72,96**	73,29	2,25	0,24
E_5	2	7 val · con BC	3	**73,35**	74,40	3,92	0,41

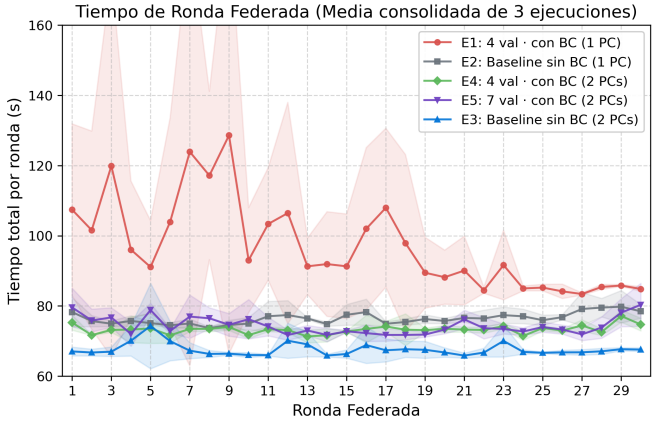


Figura 6. Comparativa del tiempo total por ronda federada en las cinco configuraciones experimentales.

El análisis multiejecución revela diferencias clave en la estabilidad de cada entorno. En la configuración local E_1, la desviación típica es sustancialmente superior ($\sigma = 27{,}64$ s) debido a que los procesos de entrenamiento convolucional compiten directamente en CPU y memoria con los ocho contenedores de Docker que soportan los nodos de Besu y el daemon de IPFS. Por el contrario, en el entorno distribuido (E_4 y E_5), la variabilidad se reduce drásticamente ($\sigma \in [2{,}25, 3{,}92]$ s y $SE \leq 0{,}41$ s), demostrando una elevadísima repetibilidad operativa. Por este motivo, se adopta la **mediana** como métrica central para aislar el comportamiento intrínseco del sistema frente a fluctuaciones externas.

D. Sobrecoste Temporal Aislado de la Capa Blockchain

Para cuantificar con exactitud el coste temporal impuesto por la serialización de tensores, la publicación en IPFS, la firma ECDSA y el consenso IBFT 2.0, se comparan las configuraciones equivalentes con y sin capa de seguridad (Figura 7):

- **Escenario Monomáquina (E_1 vs. E_2)**: El tiempo mediano pasa de 75,70 s a 85,42 s, representando un sobrecoste de **+9,72 s (+12,8 %)**.
- **Escenario Distribuido (E_3 vs. E_4)**: El tiempo mediano se incrementa de 66,62 s a 72,96 s, lo que supone una sobrecarga de tan solo **+6,34 s (+9,5 %)**.

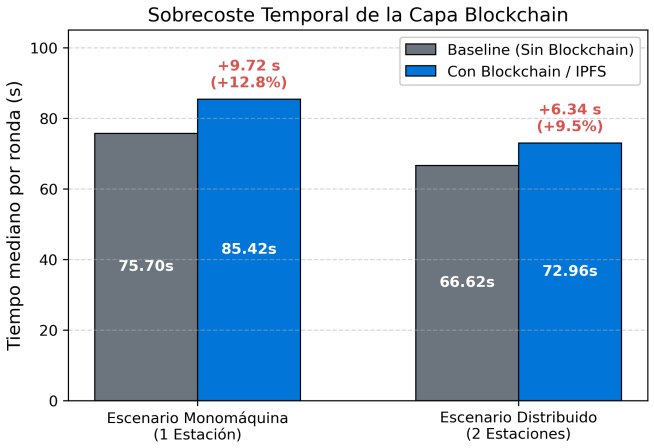


Figura 7. Sobrecoste temporal inducido por la integración de la Blockchain en entornos local y distribuido.

Este hallazgo es de especial trascendencia para aplicaciones vehiculares y de borde: al desacoplar el entrenamiento de los clientes en una estación independiente, el impacto de la capa de auditoría e inmutabilidad se sitúa por debajo del 10 %. Una penalización de apenas 6 segundos por ronda resulta ampliamente asumible frente al beneficio de contar con un registro descentralizado e incorruptible.

E. Efecto de la Distribución Física y Desglose por Fases

La comparación entre la arquitectura monomáquina y distribuida demuestra que la separación física de roles optimiza el rendimiento global (Figura 8): el tiempo de ronda se reduce tanto en el baseline sin Blockchain (de 75,70 s a

66,62 s; un descenso de -12,0 %) como con Blockchain (de 85,42 s a 72,96 s; una reducción de -14,6 %).

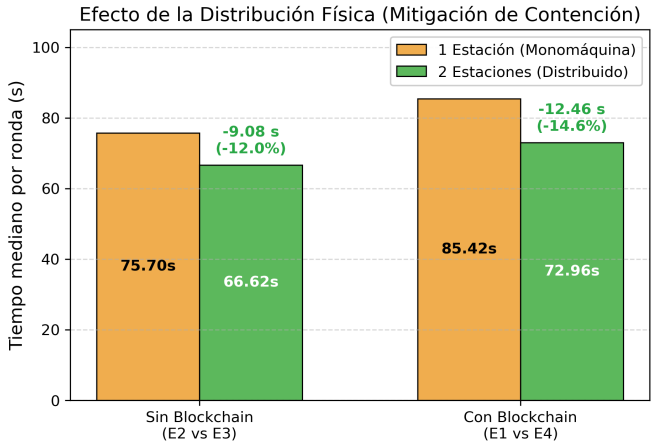


Figura 8. Efecto de la distribución física de nodos sobre el tiempo de ciclo: mitigación de cuellos de botella de CPU/memoria.

Para explicar rigurosamente este comportamiento, la Tabla V descompone la latencia media de cada fase interna del ciclo federado tanto en los clientes (Estación 2) como en el servidor (Estación 1).

Cuadro V
Desglose fino de la latencia media por fase en cliente y servidor (segundos).

	Fases Cliente (Estación 2)				Fases Servidor (Estación 1)			
Exp.	**Train**	**Verif.**	**IPFS**	**Tx Besu**	**Espera**	**Verif.**	**FedAvg**	**Bloque**
E_1	94,08	0,08	0,06	2,85	98,16	0,05	0,0065	5,14
E_2	76,45	0,00	0,00	0,00	77,02	0,00	0,0040	0,00
E_3	67,62	0,00	0,00	0,00	71,07	0,00	0,0036	0,00
E_4	66,22	1,68	2,43	2,84	79,54	0,06	0,0047	2,89
E_5	65,40	1,91	3,93	2,94	82,63	0,06	0,0079	2,99

Los datos de la Tabla V confirman de forma concluyente dos principios:

1. **Desacoplamiento de cómputo:** En las configuraciones distribuidas (E_3, E_4, E_5), el tiempo dedicado exclusivamente a la optimización neuronal con PyTorch se mantiene estrictamente invariante ($\sim 65,4$–$67,6$ s), eliminando la contención de recursos observada en E_1 ($94,08$ s).

2. **Sobrecarga de red acotada:** La verificación remota vía RPC ($1,68$–$1,91$ s) y la transferencia de pesos mediante IPFS ($2,43$–$3,93$ s) sobre Wi-Fi añaden una latencia mínima, mientras que el cálculo de FedAvg en el servidor es despreciable (< 8 ms).

F. Escalado de Consenso Bizantino IBFT 2.0 (4 frente a 7 Validadores)

En redes permisionadas, la tolerancia a fallos bizantinos sigue el principio $N \geq 3f + 1$, donde N es el conjunto de validadores y f el número máximo de nodos bizantinos o caídos:

- Red de 4 validadores (E_4): tolera $f = 1$ fallo; quórum de firmas necesario $2f + 1 = 3$.

- Red de 7 validadores (E_5): tolera $f = 2$ fallos; quórum de firmas necesario $2f + 1 = 5$.

Al comparar E_4 con E_5, la mediana por ronda apenas varía de 72,96 s a 73,35 s (+0,39 s), lo que evidencia que la latencia global del ciclo sigue dominada por el entrenamiento local.

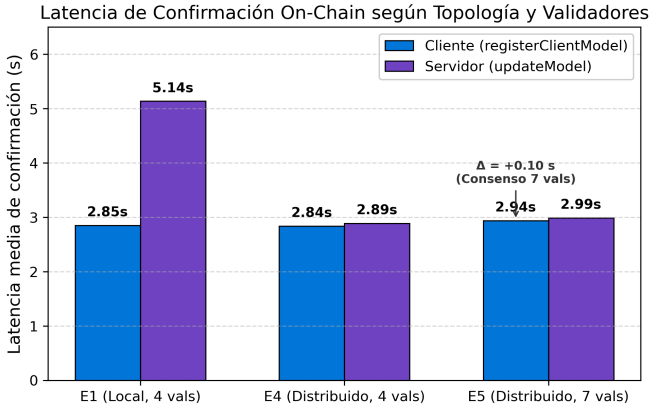


Figura 9. Tiempo medio de confirmación de transacción según topología y número de validadores en Hyperledger Besu.

Al aislar específicamente el **tiempo de confirmación de las transacciones on-chain** (Figura 9), la complejidad de intercambio de mensajes del algoritmo IBFT 2.0 ($\mathcal{O}(N^2)$ en fases *Prepare* y *Commit*) se manifiesta con absoluta coherencia:

- Transacción del cliente (`registerClientModel`): la latencia media pasa de 2,84 s en E_4 a 2,94 s en E_5 (+0,10 s).

- Transacción del servidor (`updateModel`): la latencia pasa de 2,89 s en E_4 a 2,99 s en E_5 (+0,10 s).

Este incremento simétrico de 100 ms confirma experimentalmente que ampliar la resiliencia bizantina del consorcio de 1 a 2 nodos fallidos no compromete en absoluto la viabilidad temporal del sistema.

G. Robustez Adversarial y Verificación Experimental de Ataques

Para validar empíricamente que la arquitectura no solo registra transacciones en condiciones nominales, sino que neutraliza activamente vectores de ataque reales, se implementó una batería de ensayos adversariales controlados en el módulo `test_verificacion.py`. El objetivo es evaluar el comportamiento del sistema ante las dos principales amenazas bizantinas en aprendizaje federado no confiable: un servidor central comprometido (ataque descendente) y un nodo participante envenenador (ataque ascendente).

1) Prueba 1: Verificación Descendente (Servidor Adulterado frente a Cliente Edge): En un entorno convencional, si el servidor agregador o un intermediario en red (*Man-in-the-Middle*) altera los pesos del modelo global antes de distribuirlos, los clientes entrenan de forma inadvertida sobre parámetros maliciosos. Para impedirlo, el código implementa una rutina de protección en dos fases:

1. **Generación y Sellado Oficial:** El servidor legítimo instancia el modelo base `NetCliente`, serializa los tensores mediante `pickle.dumps` y los publica en el daemon IPFS local a través del endpoint `/api/v0/add?pin=false`, obteniendo el CID oficial `QmSEBaqx...`. A continuación, el servidor emite una transacción firmada en la EVM invocando `updateModel(round, cid_oficial)` desde su cuenta (`0xfe3b55...`), quedando el compromiso criptográfico sellado en el **Bloque 15**.

2. **Inyección Adversarial y Detección Local:** Se emula un servidor malicioso que perturba los parámetros inyectando ruido aditivo ($\mathbf{w}_{\text{cand}} = \mathbf{w}_{\text{oficial}} + 0{,}5$) y remite este vector corrupto al cliente edge vía gRPC.

3. **Comprobación Cruzada y Aborto Inmediato:** Al recibir la orden de entrenamiento, el cliente edge invoca la función de solo lectura `getLastModel()` en el Smart Contract mediante una llamada JSON-RPC para recuperar el CID sellado on-chain. Con este identificador descarga de forma paralela el artefacto binario de referencia desde la pasarela IPFS (`/ipfs/<CID>`) y ejecuta una comparación bit a bit tensor por tensor (`np.allclose(p_recibido, p_ipfs, atol=1e-7)`).

Verificación Descendente en el Cliente Edge

1. **Entrada:** Pesos candidatos $\mathbf{w}_{\text{cand}}$ remitidos vía gRPC por el servidor.
2. Consultar contrato en Besu: $\text{CID}_{\text{onchain}} \leftarrow$ `FLRegistry.getLastModel().cid`
3. Descargar binario de IPFS: $\mathbf{w}_{\text{ipfs}} \leftarrow$ `IPFS.get`$(\text{CID}_{\text{onchain}})$
4. **Si** $\exists l : \|\mathbf{w}_{\text{cand},l} - \mathbf{w}_{\text{ipfs},l}\|_\infty > \epsilon$ **entonces**
   - Registrar alerta: *«Discrepancia detectada en pesos recibidos»*
   - **ABORTAR** ronda localmente (bloquear backpropagation)
5. **Si no:** Aceptar $\mathbf{w}_{\text{cand}}$ y proseguir con el entrenamiento local.

La Figura 10 ilustra la telemetría real en tiempo de ejecución capturada en la consola del cliente. El proceso íntegro de consulta RPC a Besu, descarga y cotejo tensorial insumió tan solo **0,0740 segundos** (74 ms). Al constatar la discrepancia, el cliente detuvo de forma inmediata la ejecución, impidiendo la contaminación de su modelo local y salvaguardando los datos de entrenamiento privados del vehículo.

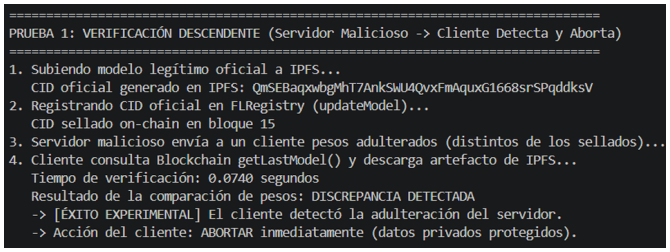


Figura 10. Prueba 1: Registro de consola del nodo cliente detectando la adulteración del modelo frente al CID oficial sellado en el bloque

Por su parte, la Figura 11 aporta la evidencia inmutable en la cadena de bloques, capturada desde el explorador de la red Hyperledger Besu. En ella se observa el sellado de la transacción `updateModel` en el **Bloque 15**, emitida por la dirección del servidor (`0xfe3b557e...`) con hash criptográfico `0x0af3ebfb...` y un consumo de gas de 167.460 (límite 500.000). Esta correspondencia unívoca entre el bloque minado y la verificación del cliente valida la consistencia del protocolo.

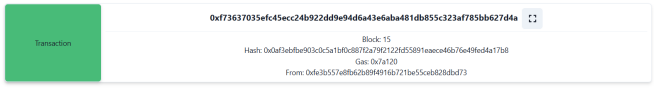


Figura 11. Prueba 1: Confirmación en Hyperledger Besu del bloque 15, registrando la transacción oficial `updateModel` emitida por el servidor agregador.

2) Prueba 2: Verificación Ascendente (Cliente Envenenador frente a Servidor Agregador): La segunda vulnerabilidad crítica consiste en la inyección de gradientes o pesos corruptos por parte de un nodo cliente malicioso (*model poisoning*). Para neutralizarla, el servidor exige que toda contribución reportada esté previamente autenticada y sellada en la Blockchain:

1. **Registro Legítimo On-Chain:** El nodo cliente honesto (cuenta 3, dirección `0xf17f52...`) publica sus pesos en IPFS y emite una transacción firmada con su clave privada invocando `registerClientModel(round, cid_legitimo)`, la cual queda consolidada por los validadores en el **Bloque 16**.

2. **Inyección Adversarial:** Se emula un nodo malicioso que intenta colar una actualización envenenada remitiendo al servidor un identificador arbitrario no respaldado on-chain (`QmClientFraudeQueNoEstaRegistradoEnBesu...`).

3. **Auditoría RPC y Descarte en FedAvg:** Antes de procesar la agregación matricial de la ronda, el servidor ejecuta una llamada de solo lectura JSON-RPC `getClientModel(round, address)` sobre el contrato `FLRegistry`. Al comparar el valor reportado con el registro inmutable firmado, se detecta la no coincidencia de hashes.

Auditoría RPC Ascendente en el Servidor Agregador

1. **Entrada:** Identificador $\text{CID}_{\text{rep}}^{(k)}$ reportado por el cliente k con dirección addr_k.
2. Consultar registro del cliente en Besu: $\text{CID}_{\text{onchain}}^{(k)} \leftarrow \texttt{getClientModel}(t, \text{addr}_k)$
3. **Si** $\text{CID}_{\text{rep}}^{(k)} \neq \text{CID}_{\text{onchain}}^{(k)}$ **entonces**
   - Registrar alerta: *«Discrepancia detectada / Fraude»*
   - **DESCARTAR** contribución del cliente k del pool de FedAvg
4. **Si no:** Incorporar pesos de k en la agregación global.

La Figura 12 documenta la salida en terminal del servidor agregador. La llamada JSON-RPC a la EVM tomó únicamente **0,0439 segundos** (44 ms). Al constatar que el CID no coincidía con la transacción firmada del cliente, el servidor excluyó al participante de la agregación de forma inmediata, preservando la convergencia y la integridad del modelo global.

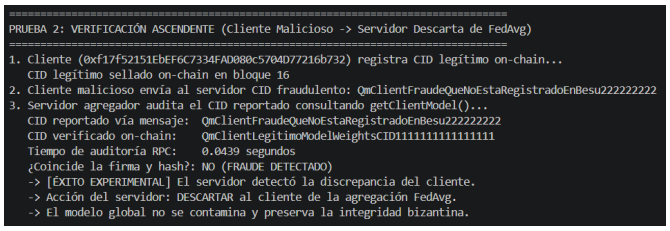


Figura 12. Prueba 2: Registro de consola del servidor agregador auditando vía RPC el CID del cliente y descartando la contribución fraudulenta

La Figura 13 muestra el recibo del **Bloque 16** en el explorador Besu, donde consta la transacción `registerClientModel` emitida por la cuenta del cliente (`0xf17f5215...`) con hash `0x3e15209c...` y un consumo de gas de 160.200 (límite 200.000). Dado que el atacante remitió un CID ajeno al sellado en dicho bloque, la verificación cruzada interceptó la inyección adversarial gracias a la resistencia a colisiones de la función de resumen SHA-256.

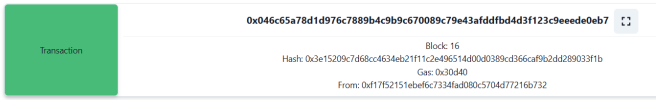


Figura 13. Prueba 2: Confirmación forense en Hyperledger Besu del bloque 16, registrando la transacción legítima `registerClientModel` del nodo cliente.

La Tabla VI sintetiza las métricas cuantitativas obtenidas en ambas pruebas de penetración adversarial.

CUADRO VI
EVALUACIÓN EMPÍRICA DEL PROTOCOLO DE DOBLE VERIFICACIÓN FRENTE A LOS VECTORES DE ATAQUE ADVERSARIAL EVALUADOS.

Vector de Ataque	Entidad Defensora	Tasa Detección	Latencia	Impacto Modelo Global
Servidor Adulterado	Cliente (Edge)	**100,0 %**	0,074 s	Aborto local; envenenamiento nulo
Cliente Malicioso	Servidor Agregador	**100,0 %**	0,044 s	Descarte inmediato; FedAvg íntegro

Estos resultados demuestran que la sobrecarga temporal introducida por el marco confiere una garantía determinista de integridad frente a manipulaciones de pesos y modelos disidentes en tránsito, neutralizando dichos vectores con tiempos de respuesta inferiores a la décima de segundo.

Conviene delimitar con rigor el alcance de este mecanismo defensivo: la doble verificación criptográfica en cadena protege de forma exhaustiva contra la adulteración directa de parámetros (*tampering*), la suplantación de identidad y la inyección de modelos no autorizados on-chain. No obstante, si un cliente malicioso genera una actualización perjudicial mediante manipulación local de sus datos de entrenamiento (*data poisoning* o *label flipping*) y la firma legítimamente con su clave privada en la Blockchain, el compromiso criptográfico coincidirá formalmente pero el

contenido neuronal estará contaminado. La mitigación de esta tipología de ataques semánticos requiere complementar la trazabilidad con filtros de agregación estadística bizantina (como Krum, Median o Trimmed-Mean) coordinados desde el contrato, aspecto que se analiza formalmente en las líneas futuras.

H. Consumo y Previsibilidad del Gas en la EVM

La Figura 14 ilustra el consumo de gas computacional de las operaciones registradas a lo largo de las rondas.

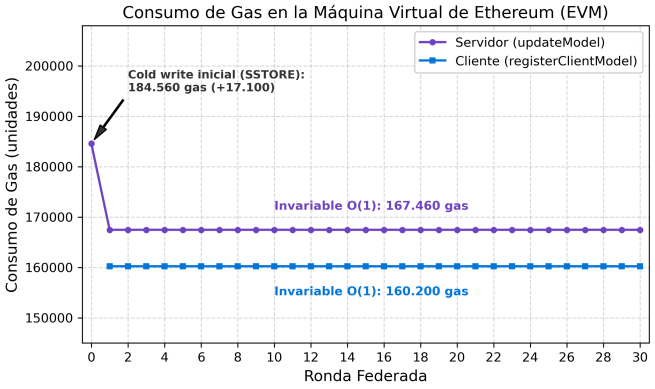


Figura 14. Consumo de gas en la Máquina Virtual de Ethereum por tipo de transacción a lo largo de las 30 rondas.

El perfil de consumo resulta rigurosamente constante en todas las iteraciones:

- `registerClientModel`: exactamente **160.200 unidades de gas**.
- `updateModel`: exactamente **167.460 unidades de gas**.

La única excepción observable corresponde a la ronda 0 del servidor, que demandó **184.560 unidades de gas**. Dicha discrepancia (+17.100 unidades) responde al coste de la instrucción `SSTORE` de la EVM al escribir por primera vez en posiciones de memoria previamente vacías (*cold write*) y modificar el valor del flag booleano `hasModel`.

La invariabilidad del consumo valida formalmente el diseño desacoplado: al mantener almacenamiento de complejidad $\mathcal{O}(1)$ en el contrato (hashes de 32 bytes y metadatos acotados), el coste en recursos computacionales es determinista e independiente del tamaño del modelo neuronal o del número de rondas ejecutadas, garantizando una total previsibilidad operativa en redes de consorcio.

I. Auditoría de Inmutabilidad On-Chain y Trazabilidad Forense

Para verificar empíricamente la correspondencia unívoca entre los eventos del ciclo de vida de Flower y la persistencia inmutable en la cadena de bloques, la Tabla VII recoge una muestra representativa de los recibos de transacción (*transaction receipts*) extraídos de la telemetría en tiempo de ejecución, correlacionada directamente con la visualización en el explorador de bloques de la red de consorcio (Figura 15).

Cuadro VII
Muestra de trazabilidad y auditoría forense on-chain.

Bloque	Entidad	Evento / Método	Dirección Emisora	Prefijo Tx Hash	Gas EVM
8	Servidor	Despliegue `FLRegistry`	`0xfe3b557e...`	`0x939eb46c...`	1.999.000
13	Servidor	`updateModel`	`0xfe3b557e...`	`0xa18e3bb4...`	167.460
14	Cliente 2	`registerClientModel`	`0x62730609...`	`0xc1c7402f...`	160.200
765	Servidor	`updateModel` (Ronda 23)	`0xfe3b557e...`	`0x4ceabb7a...`	167.460
783	Servidor	`updateModel` (Ronda 24)	`0xfe3b557e...`	`0xb9cb75f5...`	167.460
801	Servidor	`updateModel` (Ronda 25)	`0xfe3b557e...`	`0xdf69295f...`	167.460

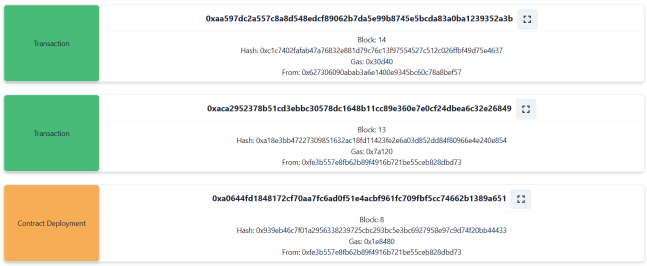


Figura 15. Auditoría de inmutabilidad on-chain: visualización en el explorador de bloques de Hyperledger Besu del despliegue del contrato inteligente `FLRegistry` (tarjeta inferior en amarillo, Bloque 8) por la dirección del servidor (`0xfe3b55...`) junto a los bloques subsiguientes de transacciones operativas 13 y 14.

Cada identificador de bloque y hash criptográfico actúa como una prueba forense no repudiable de la autoría y momento exacto de emisión de cada iteración, confirmando la viabilidad de la arquitectura para auditorías regulatorias en sistemas vehiculares y de misión crítica.

VI. Conclusiones y Trabajo Futuro

En este artículo se ha presentado, implementado y validado experimentalmente un marco integral para garantizar la trazabilidad, integridad y auditoría criptográfica en sistemas de Aprendizaje Federado, combinando una red Blockchain permisionada (Hyperledger Besu) con almacenamiento direccionable fuera de cadena (IPFS) y el framework Flower.

A. Conclusiones Principales

Los resultados obtenidos permiten extraer las siguientes conclusiones clave:

- **Viabilidad técnica y sobrecoste temporal asumible**: La integración de la capa de trazabilidad en un despliegue distribuido en red introduce una sobrecarga temporal de tan solo el **9,5 %** (un incremento mediano de 6,34 segundos por ronda federada, pasando de 66,62 s a 72,96 s). Este sobrecoste responde afirmativamente al dilema de rendimiento vs. seguridad, demostrando que la auditabilidad de extremo a extremo es plenamente viable en sistemas de IA distribuida sin degradar la cadencia operativa.
- **Eficacia y rapidez de la doble verificación cruzada**: El diseño propuesto erradica la dependencia de un servidor central como punto único de confianza. Mediante el cotejo automático de CIDs de IPFS frente a los registros inmutables on-chain, los clientes quedan protegidos frente a modelos globales corruptos (abortando la ronda en 0,074 s), mientras que el agregador descarta de inmediato contribuciones disidentes no firmadas (en 0,044 s), alcanzando una tasa de detección del 100 % frente a los vectores de adulteración e inyección evaluados con tiempos de reacción inferiores a la décima de segundo.
- **Eficiencia y predictibilidad on-chain**: El desacoplamiento arquitectónico mediante IPFS asegura un consumo de gas estrictamente determinista y de complejidad $\mathcal{O}(1)$ (160.200 gas para clientes y 167.460 gas para el servidor), evitando cualquier saturación del estado de la EVM con total independencia del tamaño de los pesos del modelo convolucional.
- **Escalabilidad del consenso BFT**: Incrementar los nodos validadores de 4 a 7 (duplicando la tolerancia a fallos bizantinos de $f = 1$ a $f = 2$) apenas eleva la latencia de confirmación de transacciones en 0,10 segundos (100 ms) y la mediana de ronda en tan solo 0,39 s, quedando este incremento absorbido de forma imperceptible dentro del cómputo de gradientes local.

B. Limitaciones y Trabajo Futuro

Como líneas de desarrollo futuro que amplían el alcance de este trabajo, se plantean:

1. **Optimización y regularización del modelo de IA**: Dado que la meta primordial de este artículo consistió en evaluar la infraestructura y extraer métricas cuantitativas del sistema FL-Blockchain, la optimización fina de la red neuronal quedó fuera del alcance primario. Como línea futura se contempla incorporar criterios de parada temprana (*early stopping*), técnicas de regularización (Dropout, normalización de batches) y arquitecturas convolucionales avanzadas para maximizar la generalización.
2. **Mecanismos de reputación y penalización on-chain**: Explotar el historial inmutable de contribuciones para articular algoritmos de puntuación de clientes o penalización (*slashing*) económica/lógica de nodos maliciosos recurrentes.
3. **Agregación bizantina avanzada**: Complementar el esquema con estrategias de agregación robustas frente a envenenamiento no evidente (como Krum o Trimmed Mean) coordinadas desde el propio contrato.
4. **Entornos no-IID y despliegue vehicular V2X**: Evaluar la robustez del framework bajo distribuciones de datos altamente heterogéneas y desplegar nodos en hardware embarcado (*edge computing*, como NVIDIA Jetson) con conectividad celular 5G/V2X.
5. **Gestión criptográfica de claves**: Sustituir el esquema de cuentas actual por módulos de seguridad de hardware (HSM) o almacenes seguros de secretos tipo Key Vault para un despliegue conforme a estándares industriales.
6. **Control de acceso on-chain y gobernanza descentralizada**: Incorporar modificadores formales de control de acceso en el Smart Contract (`onlyServer` o esquemas de consenso multifirma y gobernanza basada en roles RBAC) para restringir directamente en el bytecode de la EVM la invocación de `updateModel`, reforzando la seguridad en despliegues con múltiples servidores agregadores o federaciones abiertas.

En síntesis, la arquitectura validada demuestra que la convergencia entre Blockchain y Aprendizaje Federado no solo es factible, sino que constituye una solución sólida y escalable para el despliegue confiable de inteligencia artificial en entornos críticos y cooperativos.

VII. Referencias

[1] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas, "Communication-efficient learning of deep networks from decentralized data", in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2017.

[2] Satoshi Nakamoto, "Bitcoin: A peer-to-peer electronic cash system", `https://bitcoin.org/bitcoin.pdf`, 2008, Accedido en 2026.

[3] Zibin Zheng, Shaoan Xie, Hong-Ning Dai, Xiangping Chen, and Huaimin Wang, "Blockchain challenges and opportunities: a survey", *International Journal of Web and Grid Services*, vol. 14, no. 4, pp. 352–375, 2018.

[4] Francisco A. Pujol, Higinio Mora, Tamai Ramírez, Carlos Rocamora, and Antonio Bedón, "Blockchain-based framework for traffic event verification in smart vehicles", *IEEE Access*, vol. 12, pp. 9251–9266, 2024.

[5] CloudLab-AIA, "FL-Blockchain: Federated Learning & Blockchain for Vehicular Event Recognition", `https://github.com/cloudlab-aia/FL-Blockchain`, 2026, Repositorio en GitHub. Accedido: 2026-06-15.

[6] Daniel J. Beutel, Taner Topal, Akhil Mathur, Xinchi Qiu, Javier Fernandez-Marques, Yan Gao, Lorenzo Sani, Kwing Hei Li, Titouan Parcollet, Pedro Porto Buarque de Gusmão, and Nicholas D. Lane, "Flower: A friendly federated learning research framework", *arXiv preprint arXiv:2007.14390*, 2020.

[7] National Institute of Standards and Technology, "FIPS PUB 180-4: Secure Hash Standard (SHS)", Tech. Rep., U.S. Department of Commerce, 2015.

[8] Vitalik Buterin, "Ethereum: A next-generation smart contract and decentralized application platform", `https://ethereum.org/en/whitepaper/`, 2014, Accedido en 2026.

[9] Hyperledger Besu, "Hyperledger besu documentation", `https://docs.besu-eth.org/private-networks`, Accedido en 2026.

[10] Henrique Moniz, "The istanbul bft consensus algorithm", *arXiv preprint arXiv:2002.03613*, 2020.

[11] Ethereum Foundation, "Gas and fees", `https://ethereum.org/en/developers/docs/gas/`, Accedido en 2026.

[12] Juan Benet, "Ipfs - content addressed, versioned, p2p file system", *arXiv preprint arXiv:1407.3561*, 2014.